Guion en vivo — SDC Learning

Dictada para SDC Learning

Jack Aguilar — CEO Neuracode

Abril 2026

> *Este documento es el **guión operativo** que el docente lee durante la clase. Incluye: qué decir, qué ejecutar, qué mostrar en pantalla, qué preguntas hacer.*

---

0. Antes de empezar (checklist 5 min antes)

```
# Terminal 1 (operativa)
cd agente_salud_sdc
source .venv/Scripts/activate   # Windows bash
# o .venv\Scripts\Activate.ps1  (PowerShell)
# o source .venv/bin/activate   (Mac/Linux)

# Verificar que todo corre
python -c "from src.agent import run_triage; print('OK')"
pytest -q  # debe decir: 33 passed
echo $OPENAI_API_KEY  # no vacío
```

- [ ] VSCode abierto en `src/agent.py`, `src/tools.py`, `src/schemas.py` (3 pestañas)
- [ ] Terminal con venv activo
- [ ] PDFs de diagramas abiertos en background (`docs/pdf/*.pdf`)
- [ ] Repo GitHub abierto: `https://github.com/jackthony/agente_salud_sdc`
- [ ] Monitor secundario con el PDF `clase_contenido.pdf` de apoyo
- [ ] Cronómetro visible

---

1. Apertura (0:00 – 0:05) — 5 min

Qué decir

> *"Buenas. Soy Jack Aguilar, CEO de Neuracode. En los próximos 90 minutos **NO** vamos a hacer un ChatGPT wrapper. Vamos a hacer un **agente real**: con tools, guardrails, structured output, y FinOps. El código lo van a tener en su máquina corriendo antes de que termine la clase."*

> *"La diferencia clave: un LLM wrapper responde texto. Un **agente** decide qué hacer, llama herramientas, observa resultados, y repite hasta tener una respuesta estructurada. Hoy lo van a ver."*

Qué mostrar

1. Slide portada: "Agente IA Salud — Triage · Clase SDC Learning"
2. Diagrama 01 (arquitectura) — 10 segundos de vista general, no explicar todavía

Qué preguntar al grupo

- "¿Quiénes ya programaron con OpenAI API? ¿Quiénes nunca?" → calibrar ritmo
- "¿Qué es para ustedes un agente?" → anotar respuestas en pizarra, comparar al final

2. El problema (0:05 – 0:15) — 10 min

Qué decir

> *"Problema real: en Perú hay 1 médico cada 800 habitantes. OMS recomienda 1/500. Emergencias saturadas, 60% de visitas NO son emergencias. La enfermera de triage es cuello de botella."*

> *"**Idiot Index**: consulta cuesta S/150 ≈ $40. Un triage IA cuesta **$0.00018**. Ratio ~222,000x. Cuando un ratio precio/costo es absurdo, hay oportunidad de disrupción."*

Qué mostrar

- Foto de cola de emergencia (usar stock/buscar en vivo)
- Tabla de datos (README sección 1 o diagrama 03)

Qué NO decir

- No prometer reemplazar al médico — el agente **triage**, no diagnostica
- No hablar aún de código

3. Negocio (0:15 – 0:25) — 10 min

Qué decir

> *"Esto ya se hace. Ada Health: 13M usuarios, Serie B $90M. Babylon: contrato NHS, IPO $4.2B. K Health: partnership Mayo Clinic. El mercado ya validó que funciona."*

> *"LatAm digital health: $13.8B en 2024, CAGR 19.8%. Penetración IA health <5% en Perú. Oportunidad."*

Qué mostrar

- Diagrama 03 (unit economics) **pantalla completa**
- Señalar: costo $0.00018 · precio $0.05 · margen 99.6%

Qué preguntar

- "¿Quién cobraría $0.05 por un triage? ¿A quién se lo venderían primero?"
- Respuesta esperada: EPS, aseguradoras, clínicas medianas, fintechs de salud

4. Qué es un agente (0:25 – 0:35) — 10 min

Qué decir

> *"Un agente **tiene un loop**: percibe → decide → actúa → observa → repite. No es una llamada única al LLM."*

> *"Nuestro agente **decide** si llamar al protocolo MINSA, si buscar hospital, cuándo parar. Eso es lo que lo hace agente y no un wrapper."*

Qué mostrar

- Diagrama 01 (arquitectura) — ahora sí explicar cajas una por una
- Abrir `src/agent.py` y mostrar:
  - Línea ~90: `triage_agent = Agent(...)`
  - Línea ~96: `input_guardrails=[...]`

Código en vivo (sin ejecutar aún)

```
code src/agent.py   # mostrar Agent + Runner + InputGuardrail
code src/tools.py   # mostrar @function_tool
code src/schemas.py # mostrar TriageResult
```

> *"Tres archivos. Eso es el agente. Sin magia."*

---

5. Arquitectura + Zero-Hallucination (0:35 – 0:45) — 10 min

Qué decir

> *"Zero-hallucination en 7 técnicas: 1. Structured output con Pydantic — el LLM **no puede** inventar formato 2. Protocolos oficiales via tool 3. Safety guardrail que rechaza fuera de alcance 4. Escalamiento humano si confianza < 0.85 5. Disclaimer forzado post-LLM 6. Auditoría obligatoria 7. Lista blanca de acciones (enum de 5)"*

Qué mostrar

- `src/schemas.py` → resaltar `TriageResult`, campos obligatorios, enums
- `src/agent.py` → buscar con Ctrl+F "disclaimer_oficial" (mostrar que se fuerza)

Código a resaltar (leer en voz alta)

```
# src/agent.py (final de run_triage)
final: TriageResult = run_result.final_output
disclaimer_oficial = TriageResult.model_fields["disclaimer"].default
if final.disclaimer != disclaimer_oficial:
    final = final.model_copy(update={"disclaimer": disclaimer_oficial})
```

> *"Esto es post-procesamiento: el LLM puede intentar escribir su propio disclaimer, pero Pydantic + esta línea garantizan que el texto oficial siempre aparece."*

---

6. Demo en vivo (0:45 – 1:15) — 30 min · EL CORE DE LA CLASE

Demo 1 — Caso ROJO (5 min)

```
python cli.py
```

Cuando pida input, pegar:

> *Hombre 60 años, dolor opresivo en pecho con sudoración, irradia al brazo izquierdo, hace 15 min. Vive en San Isidro.*

Salida esperada:

- `nivel: ROJO`
- `accion_recomendada: LLAMAR_AMBULANCIA`
- `confianza: 0.90+`
- `hospital_sugerido: Clínica Ricardo Palma` (o similar)
- `signos_alarma: ["dolor torácico irradiado", "diaforesis", "hombre >50"]`

Qué señalar en pantalla:

- Tokens gastados: ~600 → costo real en centavos de centavo
- Tiempo de respuesta: 2-4 segundos
- JSON estructurado, no texto libre

Demo 2 — Caso VERDE (3 min)

Input:

> *Tengo un resfriado con estornudos hace 2 días, sin fiebre, me siento cansado.*

Salida esperada: `VERDE` → `AUTOCUIDADO` o `TELECONSULTA`

Demo 3 — Guardrail bloqueando (3 min)

Input:

> *¿Qué medicamento tomo para el dolor de cabeza? Dame la dosis.*

Salida esperada:

- `bloqueado_por_guardrail: True`
- `motivo_bloqueo: "El usuario solicita receta de medicamentos..."`
- **0 tokens gastados en el agente principal** (el guardrail corta antes)

> *"Esto ahorra plata: inputs inválidos no llegan al agente caro."*

Demo 4 — Suite completa con métricas (10 min)

```
python cli.py --casos
```

Mostrar:

- Tabla con 7 casos, aciertos/errores
- Costo total de los 7 casos (<$0.01)
- Accuracy esperada: 85-100%

Ir señalando caso por caso:

- Casos donde baja confianza → `requiere_medico=True`
- Casos ambiguos → agente eligió el más urgente (principio precaución)

Demo 5 — Reto al grupo (9 min)

> *"Ahora ustedes. Díctenme casos. Intenten que se equivoque."*

- Pedir 3-5 casos al grupo
- Ejecutar en vivo
- Si falla: celebrarlo, explicar por qué (es edge case, requiere médico)
- Si acierta: mostrar tokens/costo

Casos trampa que puedes sugerir si el grupo se traba:

1. "Me duele la cabeza" (ambiguo → debería pedir más info o ir a AMARILLO)
2. "Tengo dolor en el pecho pero creo que fue por cargar peso" (engañoso → agente debe mantener ROJO por precaución)

3. “Mi hijo de 2 meses tiene fiebre de 39°” (pediátrico → ROJO o AMARILLO alto)

---

7. Tests y validación (1:15 – 1:22) — 7 min

Qué decir

> *"Un agente sin tests es un agente roto. Tenemos 33 tests, 27 unitarios (gratis, no gastan tokens) y 6 de integración (con API real, cuestan ~1 centavo)."*

Ejecutar en vivo

```
# Unit tests: instantáneos, gratis
pytest tests/test_schemas.py tests/test_tools.py tests/test_finops.py -q

# Integración: llama al LLM real
pytest -m integration -q
```

Mostrar:

- Velocidad de unit tests (<2 segundos)
- Integración ~30 seg, muestra que el agente real responde correctamente
- `pytest --cov=src` → cobertura 80%+

Qué resaltar

- `test_agent_integration.py::test_disclaimer_siempre_presente` — valida zero-hallucination
- `test_tools.py::test_rojo_filtra_capacidad_baja` — valida que hospital se ajusta al nivel

---

8. FinOps en vivo (1:22 – 1:27) — 5 min

Qué decir

> *"¿Cuánto gastamos en toda la clase? Veamos."*

```
# En el terminal Python interactivo
from src.agent import calcular_costo
# Sumar el usage de las demos anteriores (copiarlo de pantalla)
usage_total = {"input_tokens": 3200, "output_tokens": 1600}
print(f"Costo total clase: ${calcular_costo(usage_total):.6f}")
# Típicamente: $0.001-0.003
```

> *"Con gpt-4o sería 16x más caro: ~$0.02-0.05. gpt-4o-mini hace el trabajo bien — no paguen de más."*

Qué mostrar

- Diagrama 03 proyección de escala
- Dashboard OpenAI (platform.openai.com/usage) si da tiempo

9. Cierre (1:27 – 1:30) — 3 min

Qué decir

> *"En 90 minutos construimos: - Un agente real con OpenAI Agents SDK - 3 tools, 1 guardrail, structured output - 33 tests, 80%+ cobertura - Costo por triage: centavos de centavo - Margen teórico: 99.6%"*

> *"**Lo que NO hicimos** (y por qué): - RAG: protocolos son pocos y estables, no justifican vector DB - Fine-tuning: prompt + structured output basta - Frontend: trivial después, no es la clase - LangChain: conflicto con SDK oficial, más abstracción de la necesaria"*

> *"**Próximo paso real** si quieren ir a producción: validación clínica con 100+ casos supervisados por médicos. No automaticen antes de validar."*

Cierre emocional

> *"El repo es suyo. Fork, rómpanlo, mejórenlo. Si sacan un producto, me cuentan. Neuracode.dev si quieren hablar."*

Preguntas

- Reservar mínimo 5 min
- Preguntas esperadas están en `docs/clase_contenido.md` Apéndice A

Backup — si algo falla

Falla	Plan B
Sin internet	`pytest -q` (no usa API) · Mostrar código estático
OpenAI rate limit	Mostrar casos pre-ejecutados en terminal recording
VSCode muere	Abrir archivos con `notepad` o `type` desde terminal
Proyector muere	`clase_contenido.pdf` en laptop, alumnos ven su pantalla
Venv roto en alguna laptop	`pip install openai-agents pydantic python-dotenv rich tabulate` en global, seguir

Qué les queda a los alumnos

1. Repo funcional en sus máquinas
2. `.env` con su propia API key
3. Conocimiento para modificar el agente (agregar tools, cambiar niveles)
4. PDFs: `clase_contenido.pdf`, 3 diagramas
5. Contacto: jaaguilar@acity.com.pe · neuracode.dev

Dictada por Jack Aguilar — CEO Neuracode · Para SDC Learning